



Universidad Argentina de la Empresa
Ingeniería de Datos II



Documento Inicial del Proyecto — TP Proyectual

Integrantes del grupo

Agustín Perez Leal (PM)
Franco Brunello Mioni Bonuccelli
Pilar Villaverde
Pedro Smith
Candela Masare
Facundo Martinez Zorzi

Docente: Dra. Ing. Roxana Martínez
2026



1. Descripción General del Proyecto

GameNexus es una plataforma social de gaming que combina un catálogo de juegos, una red social de jugadores, estado en tiempo real y registro de actividad histórica, utilizando cuatro motores de bases de datos distintos según la naturaleza de cada dato.

GameNexus apunta a replicar la lógica central de una plataforma tipo Steam Community: los usuarios tienen una biblioteca de juegos, una red de contactos, un estado de actividad visible y un historial de partidas. A diferencia de un launcher completo, el foco está en los datos y las relaciones entre ellos, no en la gestión de descargas o instalaciones.

La elección del dominio gaming permite justificar de forma natural la convivencia de cuatro motores con roles bien diferenciados. Ningún motor único resuelve de forma óptima todos los problemas de persistencia que plantea el sistema: hay datos con estructura variable, relaciones sociales complejas, estado volátil en tiempo real y eventos de escritura masiva. Cada motor fue elegido para el problema que mejor resuelve.

Funcionalidades principales

- Catálogo de juegos con metadata variable según género y tipo
- Biblioteca personal del usuario con horas jugadas y logros
- Red social de jugadores: amistades, grupos por juego, sugerencias
- Estado online en tiempo real: quién está jugando qué y en qué sala
- Leaderboards en vivo actualizados sin consultar bases persistentes
- Historial de partidas con estadísticas por usuario y por juego

2. Arquitectura Multi-Motor

El sistema se divide en cuatro capas de persistencia, cada una a cargo de un motor específico. Las implementadas en código Python son MongoDB y Neo4j. Redis y Cassandra se documentan con justificación técnica, esquemas y comandos representativos.

Motor	Rol en GameNexus	Implementación
MongoDB	Catálogo de juegos y biblioteca del usuario	Python con pymongo — implementado en código
Neo4j	Red social de jugadores y sugerencias de compañeros	Python con driver neo4j — implementado en código
Redis	Estado online en tiempo real y leaderboards en vivo	



MongoDB — Catálogo y biblioteca

Cada juego tiene estructura radicalmente distinta: un RPG tiene árbol de habilidades y sistema de clases; un battle royale tiene ranking de temporadas y estadísticas de partida; un juego indie puede tener solo descripción y precio. Forzar un esquema relacional implicaría decenas de columnas con NULLs masivos. MongoDB permite que cada documento refleje exactamente la estructura del juego sin imponer un molde fijo.

Además, la biblioteca personal del usuario (juegos que posee, horas jugadas por título, logros desbloqueados) se modela como subdocumento embebido en el perfil, evitando JOINS para la consulta más frecuente del sistema.

Operaciones clave:

- Búsqueda de juegos por género, plataforma o rango de precio
- Recuperación del perfil completo de un jugador con su biblioteca
- Actualización de horas jugadas y logros al cerrar sesión de juego
- Inserción de nuevos juegos con estructura variable sin alterar esquema

Neo4j — Red social y sugerencias

Las preguntas centrales del módulo social son de traversal de grafo: '¿Quiénes son los amigos de mis amigos que juegan el mismo título?', '¿Con quién debería jugar esta noche?', '¿Qué jugadores de mi red tienen logros que yo no tengo?'. En SQL estas consultas implicarían JOINS recursivos sobre tablas millonarias. En Neo4j son traversals directas sobre nodos y relaciones.

Nota técnica importante: el dataset de Steam no incluye relaciones sociales explícitas entre usuarios. El grafo se construye infiriendo afinidad a partir de juegos en común (si dos usuarios tienen muchas horas en los mismos títulos, se los conecta con una relación de afinidad ponderada). Así como también si un usuario es amigo de otro el sistema puede recomendar juegos para los dos mutuamente. Esta decisión de diseño debe documentarse y justificarse en la defensa del TP.

Operaciones clave:

- Sugerir compañeros de juego por afinidad de biblioteca
- Sugerir juegos basándose en que juegan amigos
- Detectar comunidades de jugadores por juego o género
- Calcular grado de separación entre dos jugadores
- Recomendar juegos basados en lo que juegan jugadores similares

Redis — Estado en tiempo real



Redis vive en memoria RAM y responde en milisegundos. Es el motor más crítico durante una partida activa: gestiona si el jugador está online, en qué juego y en qué sala (estado de sesión con TTL corto), el estado de la partida en curso (puntajes parciales, jugadores conectados) y los leaderboards en vivo mediante estructuras sorted sets (ZADD/ZRANGE) que se actualizan en tiempo real sin consultar Cassandra.

Sin implementación en código Python. PERO el motor tiene que funcionar y demostrar que funciona íntegramente con los demás motores en el funcionamiento del sistema. En caso de ser necesario usar un lenguaje de programación para que funcione, hacerlo.

Cassandra — Historial de partidas

Cada partida finalizada genera un registro: quién jugó, cuándo, resultado, duración, estadísticas de performance. Con miles de usuarios activos simultáneos, la escritura es masiva y continua. Cassandra escala horizontalmente sin degradar el rendimiento de escritura, particionando por jugador_id y ordenando por timestamp DESC para que el historial reciente sea siempre eficiente.

Sin implementación en código Python. PERO el motor tiene que funcionar y demostrar que funciona íntegramente con los demás motores en el funcionamiento del sistema. En caso de ser necesario usar un lenguaje de programación para que funcione, hacerlo.



3. Dataset

Dataset de Steam publicado en Kaggle por el autor Martin Bustos. Combina metadata de juegos y comportamiento de usuarios para poblar tanto MongoDB como el grafo inferido de Neo4j.

Contenido del dataset

El dataset de Martin Bustos en Kaggle contiene dos componentes principales:

- **Steam Games Dataset: metadata de juegos** — nombre, géneros, plataformas, precio, rating, descripción, fecha de lanzamiento. Alimenta directamente la colección de catálogo en MongoDB.
- **Steam User Data: comportamiento de usuarios** — juegos en biblioteca y horas jugadas por título. Sirve para construir el grafo de Neo4j infiriendo afinidad entre jugadores.

Uso por motor

Motor	Uso del dataset
MongoDB	Steam Games: documentos con metadata variable por juego. Steam User Data: biblioteca y horas como subdocumento del perfil de usuario.
Neo4j	Steam User Data: co-ocurrencia de juegos entre usuarios para inferir relaciones de afinidad y construir el grafo social.
Redis	Datos sintéticos o generados: el estado online y las sesiones activas son volátiles y no provienen del dataset histórico.
Cassandra	Datos sintéticos o generados: el historial de partidas se simula ya que el dataset no incluye eventos de partidas individuales.

4. Flujo de Datos del Sistema

Cuando un jugador interactúa con GameNexus, el sistema responde en capas según la urgencia y naturaleza de la consulta:

- Redis (milisegundos) → ¿Está online el jugador? ¿En qué partida? ¿Cuál es su posición en el ranking en vivo?
- MongoDB (consulta flexible) → ¿Qué información tiene ese juego? ¿Cuáles son sus logros? ¿Cuál es la biblioteca del usuario?
- Cassandra (escritura masiva) → Registrar el resultado de la partida finalizada y las estadísticas de performance.



- Neo4j (traversal de grafo) → ¿Con quién de su red debería jugar? ¿Qué juegos juegan jugadores con perfil similar?

Redis y Neo4j son los únicos dos motores que interactúan en secuencia: cuando el sistema sugiere compañeros de juego, Neo4j trae los candidatos por afinidad de biblioteca y Redis confirma cuáles están efectivamente online en ese momento.

5. Decisiones de Diseño y Consideraciones

Grafo inferido en Neo4j

El dataset de Steam no incluye relaciones sociales explícitas entre usuarios (no hay datos de 'A es amigo de B'). Por lo tanto, el grafo de Neo4j se construye infiriendo afinidad: si dos usuarios comparten varios juegos en su biblioteca con horas de juego significativas, se crea una relación de afinidad ponderada entre ellos. Esta decisión de diseño debe mencionarse explícitamente en la documentación y en la defensa del TP.

Justificación de la elección multi-motor

La elección de cada motor responde a la naturaleza del dato y el patrón de acceso, no a preferencia arbitraria:

- MongoDB: datos heterogéneos sin esquema fijo → documentos con estructura variable por tipo de juego
- Neo4j: problema de relaciones y traversal → grafo social donde los JOINS en SQL serían inviables
- Redis: estado volátil con latencia mínima → memoria RAM con TTL, sin persistencia duradera necesaria
- Cassandra: escritura masiva y continua → escala horizontal sin degradación de throughput